

Assignment: Features and Scale-Space Theory

Signal and Image Processing

February 23, 2026

You can work on this assignment and submit your solution (report and code) as a GROUP. This assignment counts towards your grade and have to be submitted in order to pass the course. You must follow the report guidelines found in **guidelines.pdf**. The page limit for this assignment is **15 pages** including everything, i.e. illustrations and code snippets.

1. Inspecting Spectrograms

- 1.1. (1 points) Given the soundfile **progression.wav** in the samples folder, and using a window size and stride of your choice, write a python function which takes as input a monophonic sound, a window size and generates a spectrogram (a heatmap of the magnitude of the frequencies over time) of the soundbyte. You can either implement the short-time Fourier transform using the **scipy.fft** package or using **scipy.signal.ShortTimeFFT** and a window function from **scipy.signal.windows**. Apply your spectrogram function to **progression.wav**. Using this spectrogram and the table of frequencies in table 1, determine (by visual inspection) which notes are being played by finding the lowest frequency with the most signal energy.

Deliverables: Include a code snippet and the spectrogram plot in your report. Explain briefly how you produce the spectrogram. Ensure your axes represent time and frequency, and the plot is zoomed into frequencies 0 to 1000Hz. Also report the 4 notes you identified and the window size you chose. Write 1-2 sentences about how the choice of window size affects the results.

Table 1: Frequency of notes in octave 4.

Note	Frequency
C	261.63 Hz
D	293.66 Hz
E	329.63 Hz
F	349.23 Hz
G	392.00 Hz
A	440.00 Hz
B	493.88 Hz

2. Inverse filtering

- 2.1. (1 point) Write a program that takes an image, a kernel and a realisation (a sample) of a noise source (a noise image), and which returns the result of applying the linear shift invariant (LSI) degradation model to the image. We will use the resulting image for testing in the next questions.

Deliverables: Provide a code snippet and explanation of your solution. Illustrate how the program works by applying it to the `trui.png` image for different noise levels and kernels.

- 2.2. (1 point) Implement direct inverse filtering and apply it to the LSI degraded image of `trui.png` from the previous question. You can only assume that you have the degraded image and that you know the PSF function. You are to implement this yourself - not use a function from e.g. `skimage` or other packages that implement direct inverse filtering.

Deliverables: Provide a code snippet and explanation of your solution. Experiment with different noise levels (remember to try extreme levels) and provide illustrations. Discuss this approach's ability to recover the original image.

- 2.3. (1 point) Repeat the above exercise by implementing the Wiener filter - eq. (5.8-6) in Gonzales and Woods (again you are to implement this yourself and not use an implementation from e.g. `skimage` or other packages).

Deliverables: Provide a code snippet and explanation of your solution. Experiment with different settings of the constant K approximating the fraction of noise and signal power spectra in eq. (5.8-6). Also experiment with different noise levels (remember to try extreme levels). Discuss this approach's ability to recover the original image.

3. Fixed scale feature detectors

- 3.1. (1 point) Use the `skimage.feature.canny` function on the `hand.tiff` image. Try different settings for the parameters `sigma`, `low_threshold`, and `high_threshold` of the `canny` function.

Deliverables: Include a code snippet and an illustration showing the results of the different settings and explain what the effect is of each of the parameters based on these results.

- 3.2. (1 point) Use the `skimage.feature.corner_harris` function on the `modelhouses.png` image to compute a Harris corner response image (a.k.a. feature map). Try different settings for the parameters `sigma` and `k` for `method='k'` of the `corner_harris` function. Also try to fix `sigma` and change method to `method='eps'` and try different values of the `eps` parameter.

Deliverables: Include a code snippet and an illustration showing the results of the different settings and explain what the effect is of each of the parameters based on these results.

- 3.3. (1 point) Write a Python function that finds Harris corners by finding local maxima in the feature map generated by the `skimage.feature.corner_harris` function. To find the local maxima you can use the function `skimage.feature.corner_peaks`.

Deliverables: Include in the report the code for your function. Apply this function to the `modelhouses.png` image and create a figure of the 250 strongest corner points overlaid on the `modelhouses.png` image. Hint: `corner_peaks` has a parameter to automatically select N strongest points (e.g. having the largest response value). Indicate your choice of parameter settings in the caption text of the figure.

4. Scale-space blob detector

- 4.1. (1 point) The convolution of two Gaussian functions is also a Gaussian and we have that

$$G(x, y, \sigma) * G(x, y, \tau) = G(x, y, \sqrt{\sigma^2 + \tau^2}) , \quad (1)$$

where a Gaussian function is defined as

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} . \quad (2)$$

Consider the function made from this analytical expression,

$$B(x, y) = G(x, y, \sigma) . \quad (3)$$

We will use this function as a synthetic model of a blob in an image.

Using Python, make a discrete image from the function in eq. (3) for some fixed σ by sampling the x, y axes appropriately. Visually illustrate your solution and eq. (1) by calculating images from its scale-space,

$$I(x, y, \tau) = B(x, y) * G(x, y, \tau) . \quad (4)$$

Deliverables: Provide a code snippet and explanation of your solution as well as illustrations of your solution. Confirm eq. (1) visually constructing a blob image at scale $\sigma = 1$ and convolve it with a Gaussian with scale $\tau = 2$. Also construct a model blob image with scale σ corresponding to this convolution using eq. (1). Compute and visualize the difference image between these two images and comment on the result.

- 4.2. (1 point) Prove that the expression $G(x, y, \sigma) * G(x, y, \tau) = G(x, y, \sqrt{\sigma^2 + \tau^2})$ from eq. (1) is correct.

Deliverables: Mathematical proof with essential steps and explanations. Hint: You can prove this using the Fourier transform and the convolution theorem.

- 4.3. Consider the 2-dimensional scale normalized partial derivatives of order $m + n$ at scale τ ,

$$I_{x^m y^n}(x, y, \tau) = \tau^{\gamma(m+n)} \frac{\partial^{m+n} I(x, y, \tau)}{\partial x^m \partial y^n} , \quad (5)$$

where $\gamma \in \mathbb{R}$ is a parameter of the scale normalization and $I(x, y, \tau)$ is the scale space of an image. Now consider the scale normalized Laplacian of the blob image defined in eq. (3),

$$H(x, y, \tau) = I_{xx}(x, y, \tau) + I_{yy}(x, y, \tau), \quad (6)$$

Using $\gamma = 1$, solve the following:

- i. (1 point) **Deliverables:** Write the closed form expression for $H(x, y, \tau)$ including the essential steps and explanation of the development of the expression.
- ii. (1 point) Consider the point $(x, y) = (0, 0)$ and derive analytically the scale(s), τ , for which $H(0, 0, \tau)$ is extremal. A CAS tool such as Maple (or Wolfram Alpha) may be helpful, or simply solve it by hand.

Deliverables: Mathematical derivation with essential steps and explanations. Characterise these extremal point(s) in terms of being a maximum, saddle point, or minimum as a function of τ .

- iii. (1 point) **Deliverables:** Confirm your result in Python by plotting $H(0, 0, \tau)$ as a function of τ using the expression from question i. Explain what you see in the plot.
- 4.4. (2 point) Locating the maxima and minima (x, y, τ) in the scale-space of eq. (6) with $\gamma = 1$ applied to images $I(x, y)$ in general is called blob detection with scale selection. Write a Python script that detects the 150 maxima and minima in the scale-space of the `sunflower.tiff` image with the largest absolute value of eq. (6) (You may use the scikit-image function called `peak_local_max`). Indicate each detected point and corresponding detection scale τ with a dot and a circle centred on the point of detection with a radius proportional to τ . Choose different colors so you can distinguish maxima from minima.
- Deliverables:** An illustration of your results and code snippets showing essential steps in your implementation. What image structure does maxima of eq. (6) represent, and what image structure does minima represent?